

Python Fundamentals

Lab Tasks | Identifiers · Statements · Variables · Keywords · Input-Output · Indentation · Command Line Arguments

Instructions: For each task, write the Python program in a `.py` file, run it, and record the output. Where a task asks a conceptual question, answer it in a comment at the top of the file.

LAB 01

Identifiers

Objective: Understand naming rules for Python identifiers (names given to variables, functions, classes, etc.)

- Write a program that declares five valid identifiers of different kinds: a variable, a constant-style name, a function name, a class name, and a name using an underscore. Print all of them.
- Identify which of the following are valid Python identifiers and explain why the invalid ones fail:
`2value`, `value_2`, `_hidden`, `class`, `my-var`, `MyClass`, `total$`
- Write a program that demonstrates Python's case sensitivity by creating two identifiers `Marks` and `marks` holding different values, then printing both.

CHALLENGE — Write a function `is_valid_identifier(name)` that checks (without using `str.isidentifier()`) whether a given string could be a valid Python identifier, based on the rules (starts with letter/underscore, contains only letters/digits/underscores, not a keyword).

LAB 02

Keywords

Objective: Learn Python's reserved words and how they differ from identifiers.

- Write a program that imports the `keyword` module and prints: the total number of keywords in the current Python version, and the full list of keywords.
- Write a program that checks whether a user-input word is a Python keyword or not, using `keyword.iskeyword()`.
- Try creating a variable named `for = 5` and `True = 10`. Run the program, observe the error, and write down the exact error message in a comment.

CHALLENGE — Print only the *soft keywords* (e.g., `match`, `case`, `_`) separately from the hard keywords.

LAB 03

Variables

Objective: Practice declaring, assigning, and reassigning variables of different types.

- Declare variables to store your name, age, height (float), and whether you are a student (boolean). Print each with its type using `type()`.
- Demonstrate multiple assignment: assign the values `10`, `20`, `30` to three variables `a`, `b`, `c` in a single line, then assign the same value `100` to all three variables in one line.
- Write a program that swaps the values of two variables: (a) using a temporary third variable, and (b) using Python's tuple-unpacking (`a, b = b, a`).
- Demonstrate dynamic typing by assigning an integer to a variable, printing its type, then reassigning a string to the same variable and printing its type again.

CHALLENGE — Write a program that takes the radius of a circle as a variable and calculates area and circumference, storing results in appropriately named variables.

Statements

Objective: Differentiate between expressions, statements, and compound statements.

1. Write a program containing at least: one assignment statement, one conditional (`if`) statement, one loop statement (`for` or `while`), and one print statement. Label each with a comment.
2. Write a multi-line statement using the line continuation character `\`, and an equivalent one using implicit continuation with parentheses (`()`).
3. Write a single line containing multiple statements separated by a semicolon `;` (e.g., `a=5; b=10; print(a+b)`).

CHALLENGE — Explain (in a comment) the difference between a **statement** and an **expression** in Python, and give one example of each from your own code.

Input-Output Statements

Objective: Practice taking input from the user and formatting output.

1. Write a program that takes the user's name and age as input and prints: *"Hello <name>, you will turn <age+1> next year."*
2. Write a program that takes two numbers as input (as strings, then convert using `int()` / `float()`) and prints their sum, difference, product, and quotient.
3. Demonstrate three different output formatting methods for the same data (name and marks): comma-separated `print()` , `str.format()` , and f-strings.
4. Write a program that takes multiple values in a single input line separated by spaces (e.g., `10 20 30`), splits them, converts them to integers, and prints their sum.

CHALLENGE — Take a list of 3 subject marks as input on one line, calculate and print the average, formatted to 2 decimal places.

Indentation

Objective: Understand how Python uses indentation instead of braces to define blocks.

1. Write a program with an `if-else` block and deliberately introduce an inconsistent indentation error (mix tabs/spaces or misalign a line). Run it, note the `IndentationError` message, then fix it.
2. Write a nested structure: an outer `for` loop containing an inner `if-else` , correctly indented, that prints "Even" or "Odd" for numbers 1 to 10.
3. Rewrite the following pseudocode with correct Python indentation:

```
if x > 0
print positive
else
print non-positive
```

CHALLENGE — Write a program with three levels of nested indentation (e.g., `for` → `if` → `for`) that prints a pattern (like a number or star triangle).

Command Line Arguments

Objective: Learn to pass and process arguments from the command line using `sys.argv` and `argparse`.

1. Write a program `greet.py` that accepts a name as a command line argument and prints "*Hello, <name>!*". Run it as:
`python greet.py Alice`
2. Write a program that accepts two numbers as command line arguments, converts them to integers, and prints their sum. Handle the case where the wrong number of arguments is passed using `len(sys.argv)`.
3. Print the script name (`sys.argv[0]`) and the total count of arguments passed.

CHALLENGE — Rewrite Task 2 using the `argparse` module instead of `sys.argv`, with named arguments `--num1` and `--num2`, and a `--help` message.

Submission Checklist

- ☐ All `.py` files run without errors
- ☐ Each program has a comment header with task number and your name
- ☐ Challenge tasks attempted where possible
- ☐ Screenshots or copies of terminal output included (if required by instructor)